

Prevención de Fuga de Datos con Ramas Huérfanas en Git

Cuadernillo | Vol. 1

mercedev.es — 2026-04-29

El Desafío (Síntoma)

Durante la extracción de una plantilla pública (`merci-boilerplate`) a partir de un proyecto matriz privado (`mercedev.es`), se ejecutó una sincronización estricta (`rsync --delete`) para garantizar que los archivos físicos del repositorio destino estuvieran immaculados.

Sin embargo, tras empaquetar y realizar el `git push` , una auditoría reveló una fuga de datos (Data Leak). Aunque los archivos físicos privados ya no existían, el historial de Git conservaba todos los *commits* originarios de la matriz. Esto expuso metadatos privados, correos electrónicos del autor y debates arquitectónicos internos que no debían ser de dominio público. Git había actuado como un notario implacable: guardó el borrado de los archivos, pero conservó la memoria de que alguna vez existieron.

La Maniobra (Lógica)

Para aniquilar el historial sin destruir los archivos actuales del disco duro, se aplicó una técnica avanzada de Git conocida como "**Rama Huérfana**" (**Orphan Branch**).

Esta maniobra crea una línea temporal completamente nueva y desconectada, permitiendo reescribir la historia en el servidor remoto:

```
# QUÉ HACE: Crea una rama totalmente desconectada del historial anterior.
# POR QUÉ: Es la única forma nativa de Git para iniciar un historial desde cero
# manteniendo intactos los archivos actuales del directorio de trabajo.
git checkout --orphan fresh-start

# QUÉ HACE: Prepara todos los archivos immaculados para el nuevo paquete.
git add .
```

```
# QUÉ HACE: Sella el único y verdadero commit fundacional.
git commit -m "feat: Release v1.0.0 - Lanzamiento fundacional"

# QUÉ HACE: Destruye la rama principal antigua (contaminada)
localmente.
git branch -D main

# QUÉ HACE: Renombra nuestra rama huérfana para que sea la nueva rama
principal.
git branch -m main

# QUÉ HACE: Fuerza la subida al servidor remoto sobrescribiendo la
historia.
# POR QUÉ: El flag -f (force) le dice al servidor: "Olvida todo lo que
sabías,
# esta es la única realidad ahora".
git push -f origin main
```

El Aprendizaje / Deuda Técnica

En DevSecOps (Development, Security, and Operations - Desarrollo, Seguridad y Operaciones), el principio de *Zero Trust* (Confianza Cero) también se aplica al historial del código.

Sincronizar archivos limpios (el espacio físico) no purifica el espacio temporal. Un repositorio base o *Boilerplate* debe ser un lienzo en blanco absoluto. Truncar el historial asegura la sanitización total de la propiedad intelectual y previene el *Configuration Drift* (Deriva de Configuración) en proyectos derivados.

Aviso operativo: Forzar reescrituras de historial (`push -f`) es una maniobra destructiva que solo debe ejecutarse en repositorios fundacionales o ramas estrictamente personales. Hacerlo en ramas de colaboración activa destruirá el trabajo de otros desarrolladores.